```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris, load_wine
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.metrics import accuracy_score, f1_score, mean_squared_error
```

1. Custom Decision Tree (from scratch)

```
class CustomDecisionTree:
    def __init__(self, max_depth=None):
        self.max_depth = max_depth
        self.tree = None

    def fit(self, X, y):
        self.tree = self._build_tree(X, y)

    def _build_tree(self, X, y, depth=0):
        num_samples, num_features = X.shape
        unique_classes = np.unique(y)

        # Stopping conditions
        if len(unique_classes) == 1:
            return {'class': unique_classes[0]}
        if num_samples == 0 or (self.max_depth and depth >= self.max_depth):
            return {'class': np.bincount(y).argmax()}

        # Find best split
        best_info_gain = -float('inf')
        best_split = None
        for feature_idx in range(num_features):
            thresholds = np.unique(X[:, feature_idx])
            for threshold in thresholds:
                left_mask = X[:, feature_idx] <= threshold
                right_mask = ~left_mask
                left_y, right_y = y[left_mask], y[right_mask]
                info_gain = self._information_gain(y, left_y, right_y)
                if info_gain > best_info_gain:
                    best_info_gain = info_gain
                    best_split = {
                        'feature_idx': feature_idx,
                        'threshold': threshold,
                        'left_mask': left_mask,
                        'right_mask': right_mask
                    }

        if best_split is None:
            return {'class': np.bincount(y).argmax()}

        left_tree = self._build_tree(X[best_split['left_mask']], y[best_split['left_mask']], depth+1)
        right_tree = self._build_tree(X[best_split['right_mask']], y[best_split['right_mask']], depth+1)

        return {
            'feature_idx': best_split['feature_idx'],
            'threshold': best_split['threshold'],
            'left_tree': left_tree,
            'right_tree': right_tree
        }

    def _information_gain(self, parent, left, right):
        parent_entropy = self._entropy(parent)
        left_entropy = self._entropy(left)
        right_entropy = self._entropy(right)
        weighted_avg = (len(left)/len(parent))*left_entropy + (len(right)/len(parent))*right_entropy
        return parent_entropy - weighted_avg

    def _entropy(self, y):
        if len(y) == 0:
            return 0
        probs = np.bincount(y) / len(y)
        return -np.sum(probs * np.log2(probs + 1e-9))
```

```
    def predict(self, X):
        return [self._predict_single(x, self.tree) for x in X]

    def _predict_single(self, x, tree):
        if 'class' in tree:
            return tree['class']
        feature_val = x[tree['feature_idx']]
        if feature_val <= tree['threshold']:
            return self._predict_single(x, tree['left_tree'])
        else:
            return self._predict_single(x, tree['right_tree'])
```

2. Iris dataset: Custom vs Scikit-learn Decision Tree

```
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Custom tree
custom_tree = CustomDecisionTree(max_depth=3)
custom_tree.fit(X_train, y_train)
y_pred_custom = custom_tree.predict(X_test)
accuracy_custom = accuracy_score(y_test, y_pred_custom)

# Scikit-learn tree
sklearn_tree = DecisionTreeClassifier(max_depth=3, random_state=42)
sklearn_tree.fit(X_train, y_train)
y_pred_sklearn = sklearn_tree.predict(X_test)
accuracy_sklearn = accuracy_score(y_test, y_pred_sklearn)

print("\n=> Decision Tree Comparison (Iris)")
print(f"Custom Decision Tree Accuracy: {accuracy_custom:.4f}")
print(f"Scikit-learn Decision Tree Accuracy: {accuracy_sklearn:.4f}")
```

```
=> Decision Tree Comparison (Iris)
Custom Decision Tree Accuracy: 1.0000
Scikit-learn Decision Tree Accuracy: 1.0000
```

3. Ensemble Methods (Wine dataset)

```
wine = load_wine()
X_wine, y_wine = wine.data, wine.target
X_train, X_test, y_train, y_test = train_test_split(X_wine, y_wine, test_size=0.2, random_state=42)

# Decision Tree Classifier
dt_clf = DecisionTreeClassifier(random_state=42)
dt_clf.fit(X_train, y_train)
y_pred_dt = dt_clf.predict(X_test)
f1_dt = f1_score(y_test, y_pred_dt, average='macro')

# Random Forest Classifier
rf_clf = RandomForestClassifier(random_state=42)
rf_clf.fit(X_train, y_train)
y_pred_rf = rf_clf.predict(X_test)
f1_rf = f1_score(y_test, y_pred_rf, average='macro')

print("\n=> Ensemble Methods (Wine Classification)")
print(f"Decision Tree F1 Score: {f1_dt:.4f}")
print(f"Random Forest F1 Score: {f1_rf:.4f}")
```

```
=> Ensemble Methods (Wine Classification)
Decision Tree F1 Score: 0.9425
Random Forest F1 Score: 1.0000
```

4. Hyperparameter Tuning (Random Forest Classifier)

```
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5, 10]
}
```

```
grid_search = GridSearchCV(RandomForestClassifier(random_state=42),
                           param_grid, cv=3, scoring='f1_macro')
grid_search.fit(X_train, y_train)

print("\n=> Hyperparameter Tuning (Random Forest Classifier) ")
print("Best Parameters:", grid_search.best_params_)
print("Best F1 Score:", grid_search.best_score_)
```

```
=> Hyperparameter Tuning (Random Forest Classifier)
Best Parameters: {'max_depth': None, 'min_samples_split': 2, 'n_estimators': 100}
Best F1 Score: 0.9862945382658644
```

5. Regression Models (Wine dataset features → target continuous)

```
# For demonstration, use wine data but treat target as continuous regression
y_wine_reg = y_wine.astype(float)

X_train, X_test, y_train, y_test = train_test_split(X_wine, y_wine_reg, test_size=0.2, random_state=42)

# Decision Tree Regressor
dt_reg = DecisionTreeRegressor(random_state=42)
dt_reg.fit(X_train, y_train)
y_pred_dt = dt_reg.predict(X_test)
mse_dt = mean_squared_error(y_test, y_pred_dt)

# Random Forest Regressor
rf_reg = RandomForestRegressor(random_state=42)
rf_reg.fit(X_train, y_train)
y_pred_rf = rf_reg.predict(X_test)
mse_rf = mean_squared_error(y_test, y_pred_rf)

print("\n=> Regression Models (Wine) ")
print(f"Decision Tree MSE: {mse_dt:.4f}")
print(f"Random Forest MSE: {mse_rf:.4f}")

# Hyperparameter tuning with RandomizedSearchCV
param_dist = {
    'n_estimators': [50, 100, 200, 300],
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 5, 10]
}
random_search = RandomizedSearchCV(RandomForestRegressor(random_state=42),
                                   param_distributions=param_dist,
                                   n_iter=5, cv=3, scoring='neg_mean_squared_error',
                                   random_state=42)
random_search.fit(X_train, y_train)

print("\n=> Hyperparameter Tuning (Random Forest Regressor) ")
print("Best Parameters:", random_search.best_params_)
print("Best Score (Negative MSE):", random_search.best_score_)
```

```
=> Regression Models (Wine)
Decision Tree MSE: 0.1667
Random Forest MSE: 0.0648

=> Hyperparameter Tuning (Random Forest Regressor)
Best Parameters: {'n_estimators': 300, 'min_samples_split': 2, 'max_depth': 10}
Best Score (Negative MSE): -0.05090262509850276
```